03. L2 opchild 모듈 상세 분석

개요

`opchild` 모듈은 OPinit L2 체인에서 동작하는 핵심 모듈로, L1과 L2 간의 브리지 기능, 검증자 관리, 토큰 입출금 처리, 오라클 업데이트 등의 기능을 담당합니다. 이 모듈은 L1의 `ophost` 모듈과 상호작용하여 Optimistic Rollup의 L2 측 기능을 구현합니다.

Keeper 구조 분석

1. Keeper 정의

```
type Keeper struct {
cdc codec.Codec
storeService corestoretypes.KVStoreService

authKeeper types.AccountKeeper
bankKeeper types.BankKeeper

// 서명 검증 및 시퀀스 증가 데코레이터
decorators sdk.AnteHandler
txDecoder sdk.TxDecoder
msgRouter baseapp.MessageRouter
```

```
// 모듈 권한 주소 (일반적으로 x/opchild 모듈 계정)
authority  string

// 주소 코덱들
addressCodec address.Codec
validatorAddressCodec address.Codec
consensusAddressCodec address.Codec

// Collections를 이용한 상태 관리
Schema collections.Schema
NextL1Sequence collections.Sequence // L1에서 finalize된 시퀀스
NextL2Sequence collections.Sequence // L2에서 생성되는 시퀀스
Params collections.Item[types.Params]
BridgeInfo collections.Item[types.BridgeInfo]
LastValidatorPowers collections.Map[[]byte, int64]
Validators collections.Map[[]byte, types.Validator]
ValidatorsByConsAddr collections.Map[[]byte, []byte]
HistoricalInfos collections.Map[int64, cosmostypes.HistoricalInfo]
DenomPairs collections.Map[string, string] // L2 denom -> L1 base denom
MigrationInfos collections.Map[string, types.MigrationInfo]
IBCToL2DenomMap collections.Map[string, string] // IBC denom -> L2 denom

ExecutorChangePlans  map[uint64]types.ExecutorChangePlan
```

```
l2OracleHandler *L2OracleHandler

HostValidatorStore *HostValidatorStore



// 토큰 생성 함수 (첫 입금 시 토큰 생성용)

tokenCreationFn  func(ctx context.Context, denom  string, decimals  uint8) error

}
```

2. 주요 Collections 설명

2.1 시퀀스 관리

- **NextL1Sequence**: L1에서 finalize된 마지막 시퀀스 번호 + 1
- **NextL2Sequence**: L2에서 생성되는 다음 시퀀스 번호 (출금 시 사용)

2.2 브리지 정보

- **BridgeInfo**: L1 브리지와의 연결 정보 저장
- **DenomPairs**: L2 토큰과 L1 기본 토큰 간의 매핑

2.3 검증자 관리

- **Validators**: 검증자 정보 저장 (operator address -> Validator)
- **ValidatorsByConsAddr**: 합의 주소로 검증자 조회 (consensus address -> operator address)
- **LastValidatorPowers**: 마지막 검증자 파워 저장

2.4 마이그레이션

- **MigrationInfos**: 토큰 마이그레이션 정보 (L2 denom -> MigrationInfo)

- **IBCToL2DenomMap**: IBC 토큰과 L2 토큰 간 매핑

주요 메시지 타입 분석

1. 브리지 관련 메시지

1.1 MsgFinalizeTokenDeposit

L1에서 L2로의 토큰 입금을 완료하는 메시지입니다.

```
type MsgFinalizeTokenDeposit struct {
Sender  string  // 브리지 실행자 주소
From  string  // L1 송신자 주소
To  string  // L2 수신자 주소
Amount sdk.Coin // 입금할 토큰 양
Sequence  uint64  // L1 시퀀스 번호
Height  uint64  // L1 블록 높이
BaseDenom  string  // L1 기본 토큰명
Data []byte  // 추가 실행 데이터 (hook용)
}
```

동작 과정:

1. 브리지 실행자 권한 확인
2. L1 시퀀스 검증 (순차적 처리 보장)
3. 토큰 발행 및 수신자에게 전송
4. 토큰 메타데이터 및 denom pair 등록

5. Hook 데이터가 있으면 실행

6. 실패 시 자동 출금 처리로 환불

1.2 MsgInitiateTokenWithdrawal

L2에서 L1으로의 토큰 출금을 시작하는 메시지입니다.

```
type MsgInitiateTokenWithdrawal struct {
Sender  string  // 출금 요청자
To  string  // L1 수신자 주소
Amount sdk.Coin // 출금할 토큰 양
}
```

동작 과정:

1. 사용자 토큰 잔액 확인
2. 토큰을 모듈 계정으로 전송 후 소각
3. L2 시퀀스 번호 생성
4. 출금 이벤트 발생 (L1에서 처리용)

2. 검증자 관리 메시지

2.1 MsgAddValidator

새로운 검증자를 추가하는 메시지입니다.

```
type MsgAddValidator struct {
Moniker  string  // 검증자 별명
Authority  string  // 권한 주소 (모듈 authority)
```

```
ValidatorAddress  string  // 검증자 operator 주소
Pubkey *codectypes.Any // 합의 공개키
}
```

동작 과정:

1. Authority 권한 확인
2. 최대 검증자 수 제한 확인
3. 중복 검증자 확인 (operator address, consensus pubkey)
4. 합의 파라미터의 pubkey 타입 검증
5. 검증자 정보 저장

2.2 MsgRemoveValidator

기존 검증자를 제거하는 메시지입니다.

```
type MsgRemoveValidator struct {
Authority  string  // 권한 주소
ValidatorAddress  string  // 제거할 검증자 주소
}
```

동작 과정:

1. Authority 권한 확인
2. 검증자 존재 확인
3. 검증자의 ConsPower를 0으로 설정
4. 상태 변경으로 인한 자동 제거 처리

3. 브리지 설정 메시지

3.1 MsgSetBridgeInfo

L1 브리지와의 연결 정보를 설정하는 메시지입니다.

```
type MsgSetBridgeInfo struct {
Sender  string  // 브리지 실행자
BridgeInfo types.BridgeInfo // 브리지 정보
}
```

BridgeInfo 구조:

```
type BridgeInfo struct {
BridgeId  uint64  // L1에서 할당된 브리지 ID
BridgeAddr  string  // L1 브리지 컨트랙트 주소
L1ChainId  string  // L1 체인 ID
L1ClientId  string  // IBC 클라이언트 ID
BridgeConfig types3.BridgeConfig // 브리지 설정
}
```

4. 토큰 마이그레이션 메시지

4.1 MsgRegisterMigrationInfo

토큰 마이그레이션 정보를 등록하는 메시지입니다.

```
type MsgRegisterMigrationInfo struct {
```

```
Authority  string  // 권한 주소
MigrationInfo types.MigrationInfo // 마이그레이션 정보
}
```

MigrationInfo 구조:

```
type MigrationInfo struct {
Denom  string  // L2 토큰명
IbcChannelId  string  // IBC 채널 ID
IbcPortId  string  // IBC 포트 ID
}
```

4.2 MsgMigrateToken

OP 토큰을 IBC 토큰으로 마이그레이션하는 메시지입니다.

```
type MsgMigrateToken struct {
Sender  string  // 마이그레이션 요청자
Amount sdk.Coin // 마이그레이션할 토큰 양
}
```

핵심 함수 동작 분석

1. FinalizeTokenDeposit 함수

```
func (ms MsgServer) FinalizeTokenDeposit(ctx context.Context, req *types.MsgFina
```

주요 로직:

1. **권한 검증**: `checkBridgeExecutorPermission` 으로 브리지 실행자 권한 확인
2. **시퀀스 검증**: L1 시퀀스의 순차적 처리 보장
3. **토큰 입금**: `safeDepositToken` 으로 안전한 토큰 전송
4. **메타데이터 등록**: 새 토큰의 경우 메타데이터 및 denom pair 등록
5. **Hook 실행**: 추가 데이터가 있으면 `handleBridgeHook` 실행
6. **실패 처리**: 입금 실패 시 자동 출금으로 환불

2. InitiateTokenWithdrawal 함수

```
func (ms MsgServer) InitiateTokenWithdrawal(ctx context.Context, req *types.MsgI
```

주요 로직:

1. **마이그레이션 처리**: `HandleMigratedTokenWithdrawal` 로 마이그레이션된 토큰 처리
2. **토큰 소각**: 사용자 토큰을 모듈로 전송 후 소각
3. **시퀀스 생성**: `IncreaseNextL2Sequence` 로 새 L2 시퀀스 생성
4. **이벤트 발생**: 출금 이벤트로 L1 처리 트리거

3. 검증자 관리 함수들

AddValidator

- 최대 검증자 수 제한 확인

- 중복성 검사 (operator address, consensus pubkey)
- 합의 파라미터 호환성 검증
- 검증자 정보 저장 및 인덱싱

RemoveValidator

- 검증자 존재 확인
- ConsPower를 0으로 설정하여 제거 마킹
- 실제 제거는 EndBlock에서 처리

Validator 관리 시스템

1. Validator 구조

```
type Validator struct {
Moniker  string  // 검증자 별명
OperatorAddress  string  // 운영자 주소
ConsensusPubkey *codectypes.Any // 합의 공개키
ConsPower  int64  // 합의 파워 (항상 1)
}
```

2. 검증자 상태 관리

검증자 추가 과정:

1. 권한 확인 (authority만 가능)
2. 최대 검증자 수 확인

3. 중복 검증자 확인

4. 공개키 타입 검증

5. 검증자 정보 저장

검증자 제거 과정:

1. 권한 확인

2. 검증자 존재 확인

3. ConsPower를 0으로 설정

4. EndBlock에서 실제 제거

3. 검증자 파워 관리

OPinit에서는 모든 검증자가 동일한 파워(1)를 가집니다:

```
func (v Validator) ConsensusPower() int64 {
return  v.ConsPower  // 항상 1
}
```

BridgeInfo 구조 분석

1. BridgeInfo 필드 설명

```
type BridgeInfo struct {
BridgeId  uint64  // L1에서 할당된 고유 브리지 ID
BridgeAddr  string  // L1 브리지 컨트랙트 주소
L1ChainId  string  // L1 체인 ID
```

```
L1ClientId  string  // IBC 클라이언트 ID (검증자 세트 검증용)
BridgeConfig types3.BridgeConfig // 브리지 설정 (제출 간격, finalization 기간 등)
}
```

2. BridgeInfo 검증 로직

```
func (info BridgeInfo) Validate(ac address.Codec) error {
if  info.BridgeId == 0 {
return  ErrInvalidBridgeInfo.Wrap("bridge id cannot be zero")
}
if  len(info.BridgeAddr) == 0 {
return  ErrInvalidBridgeInfo.Wrap("bridge address cannot be empty")
}
if  err := info.BridgeConfig.ValidateWithNoAddrValidation();  err != nil {
return  ErrInvalidBridgeInfo.Wrap(err.Error())
}
return  nil
}
```

3. BridgeInfo 설정 과정

1. **브리지 실행자 권한 확인**
2. **기존 정보와의 일관성 확인**:

- BridgeId 일치 확인
- BridgeAddr 일치 확인
- L1ChainId 일치 확인
- L1ClientId 일치 확인 (기존에 설정된 경우)

3. **BridgeInfo 저장**
4. **이벤트 발생**

권한 관리 시스템

1. 권한 유형

Authority 권한

- 검증자 추가/제거
- 파라미터 업데이트
- 관리자 변경
- 마이그레이션 정보 등록

Bridge Executor 권한

- 토큰 입금 완료
- 브리지 정보 설정
- 오라클 업데이트

Admin 권한

- 메시지 실행 (ExecuteMessages)
- 수수료 화이트리스트 관리
- 브리지 실행자 관리

2. 권한 확인 함수들

```
// 관리자 권한 확인
func (ms MsgServer) checkAdminPermission(ctx context.Context, sender  string) er


// 브리지 실행자 권한 확인
func (ms MsgServer) checkBridgeExecutorPermission(ctx context.Context, sender  s
```

토큰 관리 시스템

1. 토큰 생성 및 메타데이터

새로운 토큰이 처음 입금될 때:

1. **토큰 메타데이터 설정**: `setDenomMetadata`
2. **Denom Pair 등록**: L2 denom과 L1 base denom 매핑
3. **토큰 생성**: `tokenCreationFn` 호출 (설정된 경우)

2. 토큰 마이그레이션

OP 토큰을 IBC 토큰으로 마이그레이션:

1. **마이그레이션 정보 등록**: `MsgRegisterMigrationInfo`
2. **IBC denom 매핑**: IBC denom과 L2 denom 매핑 생성
3. **토큰 마이그레이션**: `MsgMigrateToken` 으로 실제 마이그레이션 실행

오라클 시스템

1. L2OracleHandler

L2 오라클 핸들러는 L1의 검증자 세트 업데이트를 처리합니다:

```
type L2OracleHandler struct {
keeper *Keeper
oracleKeeper types.OracleKeeper
logger log.Logger
}
```

2. 오라클 업데이트 과정

1. **권한 확인**: 브리지 실행자 권한 필요
2. **오라클 활성화 확인**: `BridgeConfig.OracleEnabled` 확인
3. **오라클 데이터 적용**: `ApplyOracleUpdate` 호출
4. **이벤트 발생**: 오라클 업데이트 이벤트

에러 처리

1. 주요 에러 타입

```
var (
ErrInvalidSequence = errors.Register(ModuleName, 2, "invalid sequence")
ErrInvalidAmount = errors.Register(ModuleName, 3, "invalid amount")
ErrNonL1Token = errors.Register(ModuleName, 4, "non-l1 token")
ErrInvalidBridgeInfo = errors.Register(ModuleName, 5, "invalid bridge info")
```

```
ErrMaxValidatorsExceeded = errors.Register(ModuleName, 6, "max validators exceed

ErrValidatorOwnerExists = errors.Register(ModuleName, 7, "validator owner exists

ErrValidatorPubKeyExists = errors.Register(ModuleName, 8, "validator pubkey exis

ErrOracleDisabled = errors.Register(ModuleName, 9, "oracle disabled")

)
```

2. 안전한 토큰 처리

```
func (ms MsgServer) safeDepositToken(ctx context.Context, toAddr sdk.AccAddress,
```

토큰 입금 시 실패 가능성을 고려한 안전한 처리:

- 수신자 주소 검증
- 토큰 발행 및 전송
- 실패 시 롤백 및 에러 메시지 반환

이벤트 시스템

1. 주요 이벤트 타입

```
const (

EventTypeFinalizeTokenDeposit = "finalize_token_deposit"

EventTypeInitiateTokenWithdrawal = "initiate_token_withdrawal"

EventTypeAddValidator = "add_validator"

EventTypeRemoveValidator = "remove_validator"
```

```
EventTypeSetBridgeInfo = "set_bridge_info"

EventTypeUpdateOracle = "update_oracle"

EventTypeMigrateToken = "migrate_token"

)
```

2. 이벤트 속성

각 이벤트는 관련 정보를 속성으로 포함:

- 시퀀스 번호
- 송신자/수신자 주소
- 토큰 정보 (denom, amount)
- 성공/실패 여부
- 실패 사유

결론

`opchild` 모듈은 OPinit L2 체인의 핵심 기능을 담당하는 복합적인 모듈입니다. 브리지 기능, 검증자 관리, 토큰 처리, 오라클 업데이트 등 다양한 기능을 통합적으로 제공하며, 각 기능은 적절한 권한 관리와 에러 처리를 통해 안전하게 구현되어 있습니다. 특히 L1과의 상호작용에서 시퀀스 기반의 순차 처리와 실패 시 자동 복구 메커니즘을 통해 시스템의 안정성을 보장합니다.